

异形圆管下料优化问题

摘 要

本文围绕异形圆管下料中的多规格母材选择、端部共切收益和连续批次余料流转问题，建立了“几何轮廓预处理-序列化下料模式-滚动时域余料前馈”的优化模型。首先由附件点云数据提取 10 类工件的轴向占用长度，并将左右端部轮廓离散为 360 个角度方向上的包络函数，从而计算 LL, LR, RL, RR 四类端部组合的共切收益矩阵。在此基础上，问题一建立多规格一维下料模型，得到总母材长度为 100000 mm、总体利用率为 98.9712% 的方案。

针对问题二，将工件正向和翻转状态作为图节点，将相邻端部拼接收益作为有向边权，在保持问题一每根母材工件分配不变的前提下重排工件顺序，得到总共切收益 2400.595 mm。针对问题三，进一步将工件分配与排序联合优化，建立以母材长度、共切收益、切换次数为词典序目标的序列化下料模式模型，得到总母材长度 97000 mm、总共切收益 2435.403 mm、总体利用率 99.5214% 的方案。

针对问题四，将长度不小于 200 mm 的剩余材料作为连续余料实体，建立滚动时域前馈模型。每个批次优先调用上一批次余料，再补充新标准母材，并逐段记录余料来源、长度及最终状态。三批次求解结果显示，总新母材长度为 252000 mm，总共切收益为 5565.627 mm，拼接后总轴向占用长度为 251345.683 mm，总体利用率为 99.7404%。所有下料方案、收益矩阵、拼接摘要和汇总指标已填入附件 result1.xlsx 至 result4.xlsx。

关键词： 一维下料；异形圆管；共切收益；序列依赖；余料流转；启发式优化

一 问题重述

某企业需要对 10 种异形圆管工件进行切割加工。每种工件由附件中的三维坐标点描述，圆管外半径为 20 mm，内半径为 19 mm，可选标准母材长度为 9000 mm、10000 mm、11000 mm 和 12000 mm。工件端部不是平齐截面，若相邻工件端部形状可以互补，则能够通过共切减少轴向占用长度。加工后若剩余长度不小于 200 mm，则该段材料作为余料入库并在后续批次中优先使用；否则视为废料。

四个问题具有递进关系。问题一要求在每类工件 50 件的需求下，不考虑共切效应，求母材总长度尽可能小的下料方案。问题二要求在问题一工件分配不变的条件下，计算端部共切收益矩阵，并优化各母材内部工件顺序。问题三要求综合考虑母材选取、工件分配、工件顺序和共切收益，重新制定单批次下料方案。问题四给出三个连续批次的需求，要求考虑余料跨批次调用和新母材补充，给出多批次优化方案。

二 问题分析

本题属于带序列依赖收益的一维下料问题。经典一维下料问题通常只需确定每根母材上切割哪些长度的工件，可用列生成方法处理大规模切割模式集合 [1, 2]；而本题中同一组工件的排列顺序和端部朝向不同，所产生的共切收益也不同，因此一个下料模式必须表示为有序工件序列。另一方面，问题四中的余料流转使得某一批次的决策会影响下一批次可用材料集合，具有带可用余料下料问题的多期特征 [4]。

本文采用“几何预处理—序列化下料模式—滚动时域余料前馈”的求解思路。几何预处理负责从三维点云中提取轴向长度和端部轮廓，形成可直接用于优化的长度参数和共切矩阵；序列化下料模式负责描述同一根母材上的工件顺序、朝向、共切收益、实际占用长度和切换次数；滚动时域模型负责在三批次之间传递余料实体，并在每个批次中优先消耗旧余料。

三 模型假设

1. 附件给出的工件三维坐标数据准确反映工件端部几何形状，切割误差、装夹误差和热变形对共切收益的影响忽略不计。
2. 工件可沿轴向正向或反向放置。方向翻转只改变左右端部参与拼接的类型，不改变工件轴向占用长度。
3. 共切收益仅由相邻两个工件的端部轮廓决定，多件连续加工时总共切收益等于相邻拼接收益之和。
4. 每根母材或余料只沿轴向排成一条线性序列，不允许分叉、并行拼接或跨母材拼接。
5. 长度不小于 200 mm 的剩余材料可作为余料入库，下一批次最多使用一次；长度小于 200 mm 的剩余材料视为不可用废料。

四 符号说明

表 1: 主要符号说明

符号	含义
I	工件类型集合, $I = \{1, 2, \dots, 10\}$
K	标准母材类型集合, 对应长度 9000, 10000, 11000, 12000 mm
L_k	第 k 类标准母材长度
D_{it}	批次 t 中第 i 类工件需求量
ℓ_i	第 i 类工件轴向占用长度
a, b	工件端部类型, $a, b \in \{L, R\}$
Δ_{ij}^{ab}	工件 i 端部 a 与工件 j 端部 b 拼接时的共切收益
$u = (i, o)$	带方向的工件状态, 其中 $o = 0$ 表示正向, $o = 1$ 表示翻转
G_{uv}	状态 u 后接状态 v 时的共切收益
p	一种序列化下料模式, 即一根材料上的有序工件状态序列
c_p	模式 p 考虑共切后的实际占用长度
b_p	模式 p 的总共切收益
s_p	模式 p 的工件类型切换次数
R_{jt}	批次 t 开始时第 j 段余料长度

五 几何预处理与共切收益模型

5.1 轴向占用长度

对第 i 类工件, 设附件点云中所有点的轴向坐标集合为 $\{x_{im}\}$, 则工件轴向占用长度定义为

$$\ell_i = \max_m x_{im} - \min_m x_{im}. \quad (1)$$

由附件计算得到的 10 种工件长度见表 2。

表 2: 工件轴向占用长度

工件	G1	G2	G3	G4	G5
长度/mm	191.547	148.826	191.547	191.547	75.000
工件	G6	G7	G8	G9	G10
长度/mm	150.000	250.000	399.924	181.033	200.000

5.2 端部轮廓提取

用轴向中点将点云分为左端点集和右端点集。对任意点 (x, y, z) , 其圆周角为

$$\theta = \text{atan2}(z, y) \pmod{2\pi}. \quad (2)$$

将 θ 按 1 度间隔离散为 360 个角度桶。左端轮廓取

$$P_i^L(\theta) = \max\{x - \min x_i : (x, y, z) \in \text{左端点集}, \theta(x, y, z) = \theta\}, \quad (3)$$

右端轮廓取

$$P_i^R(\theta) = \max\{\max x_i - x : (x, y, z) \in \text{右端点集}, \theta(x, y, z) = \theta\}. \quad (4)$$

对于缺失角度，通过相邻角度线性插值补齐。这样，每个端部被转化为长度为 360 的轮廓向量。

5.3 共切收益计算

若工件 i 的端部 a 与工件 j 的端部 b 相邻，则两端部最多可沿轴向相互嵌入的距离受所有圆周角度的最小可嵌入量限制，因此定义

$$\Delta_{ij}^{ab} = \max\left\{0, \min_{\theta} (P_i^a(\theta) + P_j^b(\theta))\right\}. \quad (5)$$

该处理将三维端部匹配问题转化为一组一维角度方向上的包络比较，与激光切割路径中利用相邻轮廓共线减少切割空隙的思想一致 [5]。计算结果填入 result2.xlsx 的四个收益矩阵。共切收益较大的端部组合主要集中在 G8、G9 及其相邻组合，例如 $\Delta_{88}^{RR} = 39.848 \text{ mm}$ ， $\Delta_{89}^{RL} = 23.104 \text{ mm}$ ， $\Delta_{98}^{LR} = 23.104 \text{ mm}$ 。这说明合理安排 G8 和 G9 的朝向与相邻位置，是提高材料利用率的关键。

六 序列化下料模型

6.1 状态扩展与模式指标

为刻画工件翻转，定义方向集合 $O = \{0, 1\}$ 。当 $o = 0$ 时工件左端为 L 、右端为 R ；当 $o = 1$ 时工件左端为 R 、右端为 L 。记状态 $u = (i, o)$ 的左端为 $\alpha(u)$ ，右端为 $\beta(u)$ 。若状态 u 后接状态 v ，则共切收益为

$$G_{uv} = \Delta_{\tau(u)\tau(v)}^{\beta(u)\alpha(v)}, \quad (6)$$

其中 $\tau(u)$ 表示状态 u 对应的工件类型。

一个下料模式 p 定义为

$$p = (u_1, u_2, \dots, u_{n_p}), \quad u_q \in I \times O. \quad (7)$$

其总共切收益、实际占用长度和切换次数分别为

$$b_p = \sum_{q=1}^{n_p-1} G_{u_q u_{q+1}}, \quad (8)$$

$$c_p = \sum_{q=1}^{n_p} \ell_{\tau(u_q)} - b_p, \quad (9)$$

$$s_p = \sum_{q=1}^{n_p-1} \mathbf{1}_{\tau(u_q) \neq \tau(u_{q+1})}. \quad (10)$$

模式 p 在长度为 λ 的材料上可行，当且仅当 $c_p \leq \lambda$ 。

6.2 总体建模思想

原草案将本题概括为

$$\mathcal{M} = \text{DW (1D-CSP-SDST)} + \text{SF (Rolling Horizon Remnant)}.$$

其中 DW 表示 Dantzig–Wolfe 列生成分解, SDST 表示序列依赖收益, SF 表示余料状态前馈。本文保留这一主干思想: 每一种可行下料方案不是普通数量向量, 而是一条带朝向的有序工件路径; 主问题负责选择下料路径, 子问题或启发式构造器负责生成新的高质量路径; 批次之间则通过余料实体集合传递状态。该结构兼顾了母材利用、共切收益和切换次数三类指标, 其中切换次数控制与模式变化控制问题具有一致的建模动机 [3]。

6.3 单批次主问题

设 $\mathcal{P}(\lambda)$ 为长度 λ 上可行的序列化模式集合, $x_{\lambda p}$ 表示采用模式 p 的次数, a_{ip} 表示模式 p 中第 i 类工件数量。单批次综合模型可写为

$$\sum_{\lambda} \sum_{p \in \mathcal{P}(\lambda)} a_{ip} x_{\lambda p} = D_i, \quad i \in I, \quad (11)$$

$$x_{\lambda p} \in \mathbb{Z}_+. \quad (12)$$

目标采用词典序:

$$\text{lexmin} \left(\sum_{\lambda} \sum_p \lambda x_{\lambda p}, - \sum_{\lambda} \sum_p b_p x_{\lambda p}, \sum_{\lambda} \sum_p s_p x_{\lambda p} \right). \quad (13)$$

该目标保证母材长度最小具有最高优先级; 在母材长度相同的方案中, 优先选择共切收益大的方案; 若仍相同, 再选择切换次数少的方案。

6.4 受限定价子问题

为解释草案中的列生成框架, 设一列最多包含 ν 个位置, y_{qu} 表示第 q 个位置是否选择状态 u , z_{quv} 表示第 q 个位置的状态 u 是否后接状态 v 。位置唯一性与弧流约束为

$$\sum_{u \in \mathcal{V}} y_{qu} = 1, \quad q = 1, \dots, \nu, \quad (14)$$

$$\sum_{v: (u,v) \in \mathcal{A}_{\lambda}} z_{quv} = y_{qu}, \quad \sum_{u: (u,v) \in \mathcal{A}_{\lambda}} z_{quv} = y_{q+1,v}, \quad (15)$$

其中 $\mathcal{A}_{\lambda}$ 为长度 λ 下预先保留的可行相邻弧集。由此可得列的数量系数、共切收益、切换次数和占用长度:

$$a_i(y) = \sum_{q=1}^{\nu} \sum_{\tau(u)=i} y_{qu}, \quad (16)$$

$$B(y, z) = \sum_{q=1}^{\nu-1} \sum_{(u,v) \in \mathcal{A}_{\lambda}} G_{uv} z_{quv}, \quad S(y, z) = \sum_{q=1}^{\nu-1} \sum_{(u,v) \in \mathcal{A}_{\lambda}} H_{uv} z_{quv}, \quad (17)$$

$$C(y, z) = \sum_{q=1}^{\nu} \sum_{u \in \mathcal{V}} \ell_{\tau(u)} y_{qu} - B(y, z) \leq \lambda. \quad (18)$$

草案中进一步提出非可行子路径割平面与反向同构对称性破缺。本文在程序实现时没有调用商业求解器逐列精确求解，而是用上述定价思想设计启发式评分函数：优先选择高 G_{uv} 、满足剩余需求、能够填满母材且减少切换的下一状态，从而近似完成“造列”过程。

七 滚动时域多批次余料前馈模型

问题四中材料来源包括新母材和历史余料。对批次 t ，统一记材料来源集合为 $\mathcal{S}_t$ ，材料 s 的可用长度为 λ_{st} ，新增新母材成本为

$$\kappa_{st} = \begin{cases} \lambda_{st}, & s \text{ 为新母材,} \\ 0, & s \text{ 为余料.} \end{cases} \quad (19)$$

批次 t 的需求约束为

$$\sum_{s \in \mathcal{S}_t} \sum_{p \in \mathcal{P}(\lambda_{st})} a_{ip} x_{spt} = D_{it}, \quad i \in I. \quad (20)$$

每段旧余料最多使用一次：

$$\sum_{p \in \mathcal{P}(R_{jt})} x_{jpt}^R \leq 1, \quad j \in \mathcal{J}_t. \quad (21)$$

若采用模式 p 后剩余 $r = \lambda_{st} - c_p$ ，则状态更新为

$$r \geq 200 \Rightarrow r \in \mathcal{J}_{t+1}, \quad 0 \leq r < 200 \Rightarrow r \text{ 为废料}. \quad (22)$$

批次 t 的词典序目标为

$$\text{lexmin} \left(\sum_{s,p} \kappa_{st} x_{spt}, - \sum_{s,p} b_p x_{spt}, \sum_{s,p} s_p x_{spt}, W_t \right), \quad (23)$$

其中 W_t 为当前批次产生的不可用废料总长度。该目标既保证三批次新标准母材用量尽可能小，又鼓励在同等新母材长度下提高共切收益并减少切换。

为避免“把废料转移成下一批次无法使用的短余料”，草案引入了下一批次影子价值思想。若长度为 r 的余料能够在下一批次形成有用模式，则赋予其前视价值 $\eta_{t+1}(r)$ ；若不能形成任何可行模式，则该价值取 0。实际实现中采用有限列池近似：

$$\hat{\eta}_{t+1}(r) = \max \left\{ 0, \max_{q \in \mathcal{Q}_{t+1}, c_q \leq r} \sum_{i \in I} \pi_{i,t+1} a_{iq} \right\}. \quad (24)$$

该项不改变前三层目标优先级，只作为同优方案的余料管理筛选依据。

八 三阶段词典序求解框架

草案中特别强调不能直接使用静态加权和。若令

$$\min \{ \omega_1 Z_{Nt} - \omega_2 Z_{Bt} + \omega_3 Z_{St} \},$$

即使 Z_{Nt} 略有变差, 也可能被较大的共切收益抵消, 从而违背题目“母材总长度绝对优先”的要求。因此本文采用三阶段锁定策略。

第一阶段最小化新标准母材总长度:

$$Z_{1t}^* = \min \left\{ \sum_{s \in \mathcal{S}_t} \sum_{p \in \mathcal{P}_t(\lambda_{st})} \kappa_{st} x_{spt} : \sum_{s,p} a_{ip} x_{spt} = D_{it} \right\}. \quad (25)$$

第二阶段在 Z_{1t}^* 的等值面上最大化共切收益:

$$Z_{Bt}^* = \max \left\{ \sum_{s,p} b_p x_{spt} : \sum_{s,p} a_{ip} x_{spt} = D_{it}, \sum_{s,p} \kappa_{st} x_{spt} = Z_{1t}^* \right\}. \quad (26)$$

第三阶段同时锁定长度和共切收益, 最小化切换次数:

$$Z_{St}^* = \min \left\{ \sum_{s,p} s_p x_{spt} : \sum_{s,p} a_{ip} x_{spt} = D_{it}, \sum_{s,p} \kappa_{st} x_{spt} = Z_{1t}^*, \sum_{s,p} b_p x_{spt} = Z_{Bt}^* \right\}. \quad (27)$$

在工程实现中, 严格等式锁定可能因启发式列池有限导致不可行, 故采用小容差锁定, 并在候选可行方案中按 $(Z_N, -Z_B, Z_S, W)$ 字典序筛选。

九 算法设计与实现

完整精确枚举所有序列化模式会产生指数级组合, 难以在比赛时间内稳定求解。根据草案“主问题选方案, 子问题造方案, 批次间传递余料”的思想, 本文将列生成框架工程化为可复现的启发式列生成算法, 核心流程如下。

1. **几何预处理。**读取工件 CSV 文件, 计算 ℓ_i 、 P_i^L 、 P_i^R 和 Δ_{ij}^{ab} , 构造状态转移收益矩阵 G_{uv} 与切换矩阵 H_{uv} 。
2. **初始化批次状态。**令 $t = 1$, 初始化余料集合 $\mathcal{J}_1 = \emptyset$ 。对当前批次, 根据需求 D_{it} 建立新母材和余料的统一材料来源集合 $\mathcal{S}_t$ 。
3. **生成初始列池。**对每种材料长度构造单一工件重复模式、按长度贪心填充模式和同类工件集中模式, 保证限制主问题初始可行。
4. **第一阶段造列。**在当前列池中优先寻找新母材长度较小的可行模式。候选工件按剩余需求压力和填充程度评分, 使模式尽量贴近材料长度上界。
5. **第二阶段造列。**在长度不变的候选集合中, 强化相邻状态的共切收益评分, 优先选取 G_{uv} 较大的转移边, 尤其是 G8、G9 及其互补端部组合。
6. **第三阶段造列。**在长度和收益接近的候选集合中, 增加同类连续奖励, 形成 $G_i \times n$ 形式的工件块, 降低切换次数。
7. **余料同优筛选。**若多个方案在前三层目标上接近, 则优先选择能够调用旧余料、减少小于 200 mm 废料并保留对下一批次有价值余料的方案。

8. **批次状态前馈。**批次结束后，逐根材料计算剩余长度。若剩余长度不小于 200 mm，则作为余料实体加入 $\mathcal{J}_{t+1}$ ；否则记为废料。
9. **滚动求解。**令 $t \leftarrow t + 1$ ，重复上述过程直到完成 B1、B2、B3 三个批次。

算法设置固定随机种子，保证结果可复现。所有结果由 Python 脚本自动写入官方 Excel 模板。

十 计算结果

10.1 问题一结果

问题一不考虑共切收益，得到总母材长度为 100000 mm，总轴向占用长度为 98971.200 mm，总体利用率为 98.9712%，工件总切换次数为 13。结果见表 3，完整下料方案已写入 result1.xlsx。

表 3: 问题一汇总结果

总母材长度/mm	总轴向占用长度/mm	总体利用率	工件总切换次数
100000	98971.200	0.989712	13

10.2 问题二结果

问题二保持每根母材的工件分配不变，仅优化母材内部顺序和工件朝向。总共切收益为 2400.595 mm，拼接后总轴向占用长度为 96570.605 mm。由于为获得较大共切收益，需要在部分母材内部穿插 G8、G9 及其互补工件，因此切换次数较问题一有所增加。

表 4: 问题二汇总结果

总母材长度/mm	总共切收益/mm	拼接后占用/mm	总体利用率	工件总切换次数
100000	2400.595	96570.605	0.965706	42

10.3 问题三结果

问题三重新联合优化母材选取、工件分配与内部顺序，总母材长度下降到 97000 mm。虽然共切收益与问题二相近，但由于母材分配重新调整，拼接后总轴向占用长度为 96535.797 mm，总体利用率提高到 99.5214%。

表 5: 问题三汇总结果

总母材长度/mm	总共切收益/mm	拼接后占用/mm	总体利用率	工件总切换次数
97000	2435.403	96535.797	0.995214	71

10.4 问题四结果

问题四按三批次需求滚动求解，并将可用余料前馈。三批次总新标准母材长度为 252000 mm，总共切收益为 5565.627 mm，拼接后总轴向占用长度为 251345.683 mm，总体利用率为 99.7404%。其中 B1 产生的 775 mm 余料在 B2 中被调用，B2 产生的 900 mm 余料在 B3 中被调用，体现了余料优先使用原则。

表 6: 问题四汇总结果

总母材长度/mm	总共切收益/mm	拼接后占用/mm	总体利用率	工件总切换次数
252000	5565.627	251345.683	0.997404	280

表 7: 问题四各批次材料调用概况

批次	新母材编号范围	被调用余料	新产生可用余料
B1	M1-M8	无	R1-8, 长度 775 mm
B2	M9-M16	R1-8	R2-9, 长度 900 mm
B3	M17-M24	R2-9	R3-8, 长度 400 mm

十一 结果检验

11.1 需求满足检验

程序对每个问题均统计所有下料序列中的工件数量, 并与需求逐项比较。问题一至问题三均满足 10 类工件各 50 件需求; 问题四中 B1、B2、B3 分别满足附件 2 给出的 468 件、433 件和 427 件需求, 没有欠产或超产。

11.2 长度可行性检验

对每根母材或余料, 程序均检查

$$c_p \leq \lambda. \quad (28)$$

输出表中每根材料的剩余长度均非负。若剩余长度小于 200 mm, 则标记为废料或用尽; 若不小于 200 mm, 则标记为入库并进入下一批次余料集合。

11.3 物料守恒检验

对每个批次, 材料输入长度等于加工占用长度、废料长度和结转余料长度之和。以问题四为例, 新标准母材总长为 252000 mm, 拼接后占用长度为 251345.683 mm, 最终仍有 400 mm 可入库余料, 其余不可用料头约 254.317 mm, 物料平衡闭合。

十二 模型评价与改进方向

本文模型的优点在于: 第一, 直接从三维点云数据计算端部共切收益, 避免人为估计; 第二, 将工件翻转、端部组合和相邻收益统一为序列化模式, 能够解释不同问题之间目标变化的原因; 第三, 多批次模型逐段追踪余料实体, 结果表中能够清楚反映余料来源、调用和最终状态; 第四, 算法不依赖商业求解器, 具有较好的可复现性。

模型的不足在于: 启发式算法不能严格证明全局最优; 切换次数与共切收益之间存在冲突, 当前方案更偏向材料利用率和共切收益; 此外, 模型未考虑实际切割中的刀缝宽度、夹具换装时间和端部共切的加工稳定性。后续可引入列生成与分支定价方法, 或在启发式候选模式基础上构造混合整数规划精修, 以进一步降低切换次数并给出更强的最优性界限。

参考文献

- [1] Gilmore P C, Gomory R E. A linear programming approach to the cutting-stock problem[J]. Operations Research, 1961, 9(6):849-859.
- [2] Gilmore P C, Gomory R E. A linear programming approach to the cutting stock problem-Part II[J]. Operations Research, 1963, 11(6):863-888.
- [3] Haessler R W. Controlling cutting pattern changes in one-dimensional trim problems[J]. Operations Research, 1975, 23(3):483-493.
- [4] Cherri A C, Arenales M N, Yanasse H H. The one-dimensional cutting stock problem with usable leftovers: A survey[J]. European Journal of Operational Research, 2014, 236(2):395-402.
- [5] Dewil R, Vansteenwegen P, Cattrysse D. A review of cutting path algorithms for laser cutters[J]. International Journal of Advanced Manufacturing Technology, 2016, 87:1865-1884.

附录

附录 A 程序说明

本文使用 Python 编写求解程序，主要依赖 pandas、numpy、openpyxl。程序文件 contest_solver.py 可独立运行，运行后自动读取附件数据并生成 result1.xlsx 至 result4.xlsx。核心命令为：

```
1 python contest_solver.py
```

附录 B 核心程序片段

为控制附录篇幅，仅保留与建模直接对应的核心代码片段：几何预处理、共切收益计算以及多批次滚动求解主流程。完整可运行程序见同目录文件 contest_solver.py。

1. 端部轮廓提取与插值

```
1 def compute_angle(y: float, z: float) -> float:
2     return math.atan2(z, y) % (2 * math.pi)
3
4
5 def extract_end_profile(points: np.ndarray, nominal_x: float, is_l_end: bool) -> Dict[int, float]:
6     raw: Dict[int, List[float]] = {}
7     for x, y, z in points:
8         theta_deg = math.degrees(compute_angle(y, z)) % 360
9         offset = (x - nominal_x) if is_l_end else (nominal_x - x)
10        raw.setdefault(int(round(theta_deg)) % 360, []).append(float(offset))
11    return {k: max(v) for k, v in raw.items()}
12
13
14 def interpolate_profile(profile: Dict[int, float], bins: int = 360) -> np.ndarray:
15     if not profile:
16         return np.zeros(bins)
17     angles = sorted(profile.keys())
18     values = [profile[a] for a in angles]
19     if len(angles) == 1:
20         return np.full(bins, values[0])
21     ext_angles = [a - 360 for a in angles[-2:]] + angles + [a + 360 for a in angles[:2]]
22     ext_values = values[-2:] + values + values[:2]
23     return np.interp(np.arange(bins), ext_angles, ext_values)
```

2. 工件数据与共切收益矩阵计算

```
1 def load_workpiece_data() -> WorkpieceData:
2     workpieces = {}
3     for i in ITEMS:
4         csv_path = WORKPIECE_DIR / f"圆管{i}.csv"
5         data = pd.read_csv(csv_path).values
6         x_min = float(np.min(data[:, 0]))
7         x_max = float(np.max(data[:, 0]))
8         x_mid = (x_min + x_max) / 2.0
```

```

9     left_points = data[data[:, 0] < x_mid]
10    right_points = data[data[:, 0] >= x_mid]
11    workpieces[i] = {
12        "length": x_max - x_min,
13        "L": interpolate_profile(extract_end_profile(left_points, x_min, True)),
14        "R": interpolate_profile(extract_end_profile(right_points, x_max, False)),
15    }
16
17    delta: Dict[Tuple[int, int, str, str], float] = {}
18    for i in ITEMS:
19        for j in ITEMS:
20            for a in ("L", "R"):
21                for b in ("L", "R"):
22                    delta[(i, j, a, b)] = max(
23                        0.0,
24                        float(np.min(workpieces[i][a] + workpieces[j][b])),
25                    )
26
27    return WorkpieceData(
28        lengths={i: float(workpieces[i]["length"]) for i in ITEMS},
29        delta=delta,
30    )

```

3. 单批次序列构造与块模式生成

```

1  def build_co_cut_sequence(
2      wp: WorkpieceData,
3      capacity: float,
4      remaining: Dict[int, int],
5      rng: random.Random,
6      prefer_fill: bool = True,
7      top_k: int = 5,
8      same_item_bias: float = 10.0,
9  ) -> Optional[List[PieceState]]:
10     seq: List[PieceState] = []
11     used = 0.0
12
13     while True:
14         candidates: List[Tuple[float, PieceState, float, float]] = []
15         demand_total = max(1, sum(max(0, v) for v in remaining.values()))
16         for item, count in remaining.items():
17             if count <= 0:
18                 continue
19             demand_pressure = count / demand_total
20             for ori in ("N", "F"):
21                 state = (item, ori)
22                 if seq:
23                     gain = transition_info(wp, seq[-1], state)[1]
24                     add_len = wp.lengths[item] - gain

```

```

25         same_bonus = same_item_bias if item == seq[-1][0] else 0.0
26     else:
27         gain = 0.0
28         add_len = wp.lengths[item]
29         same_bonus = 0.0
30         if used + add_len <= capacity + 1e-7:
31             fill_score = add_len / max(1.0, capacity - used) if prefer_fill else 0.0
32             score = 18.0 * gain + 9.0 * demand_pressure + 2.5 * fill_score + same_bonus
33             candidates.append((score, state, add_len, gain))
34         if not candidates:
35             break
36         state, add_len, _ = weighted_choice_top(rng, candidates, top_k=top_k)
37         seq.append(state)
38         used += add_len
39         remaining[state[0]] -= 1
40
41     return seq or None
42
43
44 def best_same_item_block(
45     wp: WorkpieceData,
46     item: int,
47     count: int,
48     prefix: Optional[PieceState] = None,
49 ) -> Tuple[List[PieceState], float, float]:
50     best_seq: List[PieceState] = []
51     best_key = (float("inf"), float("inf"))
52     for start_ori in ("N", "F"):
53         for mode in ("same", "alternate"):
54             seq = []
55             for k in range(count):
56                 if mode == "same":
57                     ori = start_ori
58                 else:
59                     ori = start_ori if k % 2 == 0 else ("F" if start_ori == "N" else "N")
60                 seq.append((item, ori))
61             used, benefit = sequence_metrics(wp, seq)
62             if prefix is not None:
63                 bridge = transition_info(wp, prefix, seq[0])[1]
64                 used -= bridge
65                 benefit += bridge
66             key = (used, -benefit)
67             if key < best_key:
68                 best_key = key
69                 best_seq = seq
70     used, benefit = sequence_metrics(wp, best_seq)
71     if prefix is not None and best_seq:
72         bridge = transition_info(wp, prefix, best_seq[0])[1]

```

```

73     used -= bridge
74     benefit += bridge
75     return best_seq, used, benefit
76
77
78 def build_block_sequence(
79     wp: WorkpieceData,
80     capacity: float,
81     remaining: Dict[int, int],
82     rng: random.Random,
83 ) -> Optional[List[PieceState]]:
84     seq: List[PieceState] = []
85     used = 0.0
86
87     while True:
88         candidates = []
89         for item, rem_count in remaining.items():
90             if rem_count <= 0:
91                 continue
92             max_fit = min(rem_count, int(capacity // max(1.0, wp.lengths[item]))) + 2)
93             count_candidates = {
94                 1,
95                 2,
96                 3,
97                 max_fit,
98                 max(1, max_fit - 1),
99                 max(1, max_fit // 2),
100                 min(rem_count, 8),
101                 min(rem_count, 16),
102                 min(rem_count, 32),
103             }
104             for count in sorted(c for c in count_candidates if 1 <= c <= max_fit):
105                 block, block_used, block_benefit = best_same_item_block(
106                     wp, item, count, seq[-1] if seq else None
107                 )
108                 if not block:
109                     continue
110                 if used + block_used <= capacity + 1e-7:
111                     leftover = capacity - used - block_used
112                     pressure = count / max(1, rem_count)
113                     score = (
114                         40.0 * block_benefit
115                         + 2.5 * count
116                         + 25.0 * pressure
117                         - 0.025 * leftover
118                     )
119                     candidates.append((score, item, count, block, block_used))
120     if not candidates:

```

```

121         break
122     candidates.sort(key=lambda x: x[0], reverse=True)
123     score, item, count, block, block_used = rng.choice(candidates[: min(5, len(candidates))])
124     del score
125     seq.extend(block)
126     used += block_used
127     remaining[item] -= count
128     return seq or None

```

4. 多批次滚动求解主流程

```

1 def solve_problem4(wp: WorkpieceData, batch_demands: Dict[int, Dict[int, int]], rng: random.Random)
2     -> List[PlanRow]:
3     all_rows: List[PlanRow] = []
4     leftovers: List[Material] = []
5     next_stock_id = 1
6     for batch in (1, 2, 3):
7         rows, _ = solve_single_batch(
8             wp,
9             batch_demands[batch],
10            rng,
11            leftovers,
12            batch=batch,
13            iterations=650,
14        )
15        next_stock_id = assign_material_ids(rows, next_stock_id, batch=batch)
16        leftovers = make_leftovers(rows, batch)
17        all_rows.extend(rows)
18    return all_rows
19
20 def make_leftovers(rows: List[PlanRow], batch: int) -> List[Material]:
21     next_leftovers = []
22     for idx, row in enumerate(rows, start=1):
23         if row.leftover >= MIN_USABLE_LEFTOVER:
24             if row.source == "新母材":
25                 origin_id = row.material_id
26                 first_batch = batch
27             else:
28                 origin_id = row.origin_stock_id or row.material_id
29                 first_batch = row.first_batch
30             next_leftovers.append(
31                 Material(
32                     material_id=f"R{batch}-{idx}",
33                     length=row.leftover,
34                     source="余料",
35                     origin_stock_id=origin_id,
36                     first_batch=first_batch,
37                     parent_id=row.material_id,

```

```

38         )
39     )
40     return next_leftovers

```

```

1  def main() -> None:
2      global WP_DATA
3      RESULT_DIR.mkdir(parents=True, exist_ok=True)
4      rng = random.Random(RNG_SEED)
5
6      print("Loading workpiece geometry and batch demands...")
7      WP_DATA = load_workpiece_data()
8      batch_demands = load_batch_demands()
9      all150 = {i: 50 for i in ITEMS}
10
11     print("Solving Problem 1...")
12     p1_rows = pack_problem1(WP_DATA, all150)
13     validate_demands(p1_rows, all150, "Problem 1")
14     print_metrics("Problem 1", p1_rows)
15     write_result1(WP_DATA, p1_rows)
16
17     print("Solving Problem 2...")
18     p2_rows = reorder_fixed_assignment(WP_DATA, p1_rows, rng)
19     validate_demands(p2_rows, all150, "Problem 2")
20     print_metrics("Problem 2", p2_rows)
21     write_result2(WP_DATA, p2_rows)
22
23     print("Solving Problem 3...")
24     p3_rows, _ = solve_single_batch(WP_DATA, all150, rng, iterations=750)
25     assign_material_ids(p3_rows, 1)
26     validate_demands(p3_rows, all150, "Problem 3")
27     print_metrics("Problem 3", p3_rows)
28     write_result3(p3_rows)
29
30     print("Solving Problem 4...")
31     p4_rows = solve_problem4(WP_DATA, batch_demands, rng)
32     for batch in (1, 2, 3):
33         validate_demands([r for r in p4_rows if r.batch == batch], batch_demands[batch], f"Problem 4
           batch {batch}")
34     print_metrics("Problem 4", p4_rows, use_new_stock_only=True)
35     write_result4(p4_rows)

```

附录 C AI 工具使用详情

根据《杭州电子科技大学数学建模竞赛 AI 工具使用管理规定（2026）》，本文对 AI 工具使用情况说明如下。

1. 所用 AI 工具：DeepSeek-V4-Pro; Gemini 3.1 Pro。

2. 使用日期：2026 年 5 月 28 日至 2026 年 5 月 29 日。
3. 使用目的与环节：DeepSeek-V4-Pro 辅助建模思路整理、求解流程讨论与代码辅助；Gemini 3.1 Pro 辅助论文写作、结构调整与语言润色。
4. 关键交互摘要：围绕异形圆管下料问题的目标设定、共切收益建模、余料滚动前馈、程序实现与论文排版进行了多轮问答和修改。
5. 采纳与人工修改情况：AI 工具输出仅作为辅助材料，程序结果、模型公式、参数口径、结果表填写与论文定稿均由参赛队人工核验和修改。